SellOWL

Student Marketplace — Technical Documentation

Version 1.0 | March 2026 | UW-Madison

Built with React 18, Flask, Firebase Auth, Supabase/PostgreSQL, Cloudinary, and Groq AI

1. Project Overview

SellOWL is a university-exclusive student marketplace built for UW-Madison students. It allows students to list, buy, and sell secondhand items — furniture, electronics, textbooks, clothing, and more — within a verified campus community. Only .edu email addresses are permitted, ensuring a trusted peer-to-peer environment.

The application is deployed as a Docker container on Render, with a React frontend bundled and served directly by the Flask backend in production.

1.1 Tech Stack Summary

Layer	Technology	Purpose
Frontend	React 18 + Vite + Tailwind CSS	UI framework and build tooling
Backend	Flask 3 (Python)	REST API, session management, AI routes
Authentication	Firebase Auth	.edu-only email/password auth
Database	PostgreSQL via Supabase	All persistent data storage
Image/Video Storage	Cloudinary	Direct browser upload, CDN delivery
AI — Image Scan	Groq (Llama 4 Scout Vision)	Auto-fill listing fields from photo
AI — Price Suggest	Groq (Llama 3.1 8B)	Estimate resale price range
Deployment	Render (Docker)	Cloud hosting with auto-deploy from GitHub
Version Control	GitHub	Source code repository

2. Architecture

2.1 Repository Structure

```
sellowl/
├── Dockerfile                  # Multi-stage build: Vite → Flask
├── package.json                # Frontend dependencies
├── vite.config.js              # Vite + Tailwind + proxy to :5001
├── src/
│   ├── App.jsx                 # Root component, routing, nav
│   ├── lib/
│   │   ├── api.js              # All fetch calls to backend
│   │   ├── storage.js          # Storage provider abstraction
│   │   └── firebase.js         # Firebase client config
│   └── components/
│       ├── auth/               LoginPage, SignupPage
│       ├── marketplace/        Feed, CreateListing, Checkout, Offers
```

```
│       ├── profile/              Profile, MyProfile
│       ├── messaging/            Messages, ChatThread, Chatbot
│       └── shared/               LoadingScreen, Notifications
└── backend/
    ├── app.py                    # Flask app, all API routes
    ├── ai_provider.py            # AI provider abstraction (Groq / AWS Bedrock)
    ├── storage.js                # (frontend) Storage abstraction
    ├── firebase_auth.py          # Firebase Admin SDK init + token verify
    ├── db.py                     # PostgreSQL connection + init
    ├── config.py                 # Env var loading
    ├── requirements.txt          # Python dependencies
    └── firebase-service-account.json  # (never committed)
```

2.2 Deployment Architecture

The Dockerfile uses a two-stage build:

- Stage 1 (Node/Vite): Installs npm dependencies and runs npm run build, producing a dist/ folder of static assets. All VITE_ environment variables are injected as Docker build ARGs at this stage.
- Stage 2 (Python/Flask): Installs Python dependencies, copies the backend code and the dist/ folder from Stage 1. Gunicorn serves both the Flask API and the static React files.

Render auto-deploys on every push to the main branch on GitHub.

3. Implemented Features

3.1 Authentication (.edu Only)

All authentication is handled by Firebase Auth. The flow is:

1. User signs up/logs in with an email and password via Firebase on the frontend.
2. Firebase returns a short-lived ID token (JWT).
3. The frontend sends the token to POST /api/auth/verify.
4. Flask verifies the token using Firebase Admin SDK, checks the email ends in .edu, checks email_verified = true, then upserts the user into the PostgreSQL users table and creates a server-side session.

- Emails not ending in .edu are rejected with a 403 error.
- Unverified emails are rejected — Firebase sends a verification email on signup.

3.2 Listings — Create, Read, Delete

Any authenticated user can create, view, and soft-delete their own listings.

- Create Listing form has 9 fields: Photo/Video upload, AI Auto-fill, Title, Description, Category, Condition, Price (with AI suggestion), Neighbourhood/Landmark, and Delivery Options.
- Photos and videos are uploaded directly from the browser to Cloudinary (no backend involvement in the upload itself).
- After upload, the AI Auto-fill button sends the image URL to the backend which calls Groq Llama 4 Scout Vision to generate a title, description, category, and condition.
- The AI Price Suggest button sends the listing details to Groq Llama 3.1 8B which returns a suggested price range based on student secondhand market rates.
- Delete is a soft delete — the is_available flag is set to FALSE. The row remains in the database for future order/message history references.
- A live Preview tab in the Create Listing modal shows exactly how the listing will appear in the feed before posting.

3.3 Feed

The home feed shows all available listings pulled from the database, displayed in an Instagram-style single-column format with:

- Left sidebar (desktop): Category checkboxes, condition checkboxes, price range filter.
- Mobile: Horizontal scroll category filter bar above the feed, plus a Create New Listing button.
- Like toggle (client-side only, not persisted to DB yet).
- Click-through to seller profile from any listing.

3.4 Seller Profile

Tapping a seller's listing in the feed opens their profile page showing:

- Seller name, email, listing count.

- Instagram-style 3-column grid of all their active listings.
- Tap any grid item in default mode to open a full Listing Detail modal with image, price, description, category, condition, delivery options, neighbourhood, seller info, and Buy Now / Add to Bag buttons.
- Select Items mode enables multi-select with checkboxes and a sticky Checkout / Add to Bag bar.
- Message button to start a conversation with the seller.

3.5 My Profile

The authenticated user's own profile page shows:

- Their name, email, listing count.
- + New Listing button (opens Create Listing modal).
- 3-column grid of own listings. Hover shows a quick-delete trash icon.
- Tap any listing to open a detail modal with a Remove This Listing button at the bottom.
- Delete confirmation bottom sheet shows the item thumbnail, price, and requires an explicit confirmation tap — no browser alerts.

3.6 Real-Time Messaging

A full direct messaging system allows buyers and sellers to communicate:

- Conversations are automatically created or retrieved when a user taps Message on a profile or listing detail.
- Messages list (inbox) shows all conversations with the other user's name, last message preview, and timestamp.
- Unread indicators: gold dot on avatar with count, bold name and message text, gold timestamp, and gold background tint for unread threads.
- Nav bar Messages button shows a total unread count badge, polled every 30 seconds.
- Opening a conversation automatically marks it as read.
- ChatThread supports quick-reply options for new conversations (e.g. 'Is this still available?').
- Animated typing indicator (bouncing dots) while a message is being sent.
- Auto-scroll to newest message.

3.7 Provider Abstractions (Future-Proof)

Two abstraction layers allow swapping infrastructure providers by changing a single environment variable:

- Storage (src/lib/storage.js): VITE_STORAGE_PROVIDER=cloudinary (current) or s3 (future). CreateListing.jsx calls uploadFile() without knowing which provider is active.
- AI (backend/ai_provider.py): AI_PROVIDER=groq (current) or aws_bedrock (future). All AI routes in app.py call ai.scan_image() and ai.suggest_price() without knowing which provider is active.

4. Database

4.1 Where to Find the Tables

Database: Supabase (PostgreSQL). To view and query tables:

5. Go to supabase.com and log in.
6. Select your project (sellowl).
7. In the left sidebar: Table Editor (visual) or SQL Editor (query).
8. Database → Tables shows all table schemas.

4.2 Table Schemas

users

Created automatically on first login. Stores all authenticated users.

Column	Type	Description
id	SERIAL PRIMARY KEY	Auto-increment integer ID
firebase_uid	VARCHAR(255) UNIQUE	Firebase user UID — used to link Firebase Auth to DB
email	VARCHAR(255) UNIQUE	Must end in .edu
display_name	VARCHAR(255)	User's display name from Firebase
email_verified	BOOLEAN DEFAULT FALSE	Whether Firebase has verified the email
created_at	TIMESTAMP	Account creation time
updated_at	TIMESTAMP	Last update time

listings

Created when the listings table is first needed. Stores all product listings.

Column	Type	Description
id	SERIAL PRIMARY KEY	Auto-increment listing ID
user_id	INTEGER FK → users.id	Owner of the listing
title	VARCHAR(255) NOT NULL	Listing title
description	TEXT	Full description
price	DECIMAL(10,2)	Asking price in USD
category	VARCHAR(100)	Furniture / Electronics / Books / etc.
condition	VARCHAR(50)	Like New / Good / Fair
image_url	TEXT	Cloudinary CDN URL for the uploaded image/video

Column	Type	Description
neighbourhood	VARCHAR(255)	Pickup area / landmark
delivery_option	VARCHAR(50)	Comma-separated: pickup, delivery, or both
is_available	BOOLEAN DEFAULT TRUE	FALSE = soft deleted, hidden from feed
created_at	TIMESTAMP	When the listing was posted
updated_at	TIMESTAMP	Last modification time

conversations

Auto-created when messaging tables are first needed. One row per unique user pair.

Column	Type	Description
id	SERIAL PRIMARY KEY	Conversation ID
user1_id	INTEGER FK → users.id	Lower user ID of the pair (normalised)
user2_id	INTEGER FK → users.id	Higher user ID of the pair (normalised)
created_at	TIMESTAMP	When the conversation was started
UNIQUE	(user1_id, user2_id)	Prevents duplicate conversations between same users

messages

Stores every individual message sent in a conversation.

Column	Type	Description
id	SERIAL PRIMARY KEY	Message ID
conversation_id	INTEGER FK → conversations.id	Which conversation this belongs to
sender_id	INTEGER FK → users.id	Who sent the message
text	TEXT NOT NULL	Message content
created_at	TIMESTAMP	When the message was sent

conversation_reads

Tracks when each user last read each conversation. Used to compute unread counts.

Column	Type	Description
user_id	INTEGER FK → users.id	The user who read the conversation
conversation_id	INTEGER FK → conversations.id	Which conversation was read

Column	Type	Description
last_read_at	TIMESTAMP	When they last opened/sent in this conversation
PRIMARY KEY	(user_id, conversation_id)	One read-timestamp per user per conversation

4.3 Important Notes on Data

- All tables are auto-created by the backend on first use — no manual SQL setup is required.
- Listings are never hard-deleted. The is_available column is set to FALSE instead. To recover a listing, run: UPDATE listings SET is_available = TRUE WHERE id = <id>;
- Conversations are normalised: the user with the lower integer ID is always user1_id. This prevents duplicate rows.
- The conversation_reads table uses INSERT ... ON CONFLICT DO UPDATE, so every time a user opens a thread it's a single upsert.

5. Backend API Reference

Base URL: https://sellowl.onrender.com (production) or http://localhost:5001 (local). All endpoints return JSON. Session cookie required for authenticated routes.

5.1 Auth Routes

Method	Endpoint	Auth?	Description
GET	/api/auth/me	Yes	Returns current logged-in user (id, email, display_name)
POST	/api/auth/verify	No	Verifies Firebase ID token, creates session. Body: { idToken }
POST	/api/auth/logout	Yes	Clears the server session

5.2 Listing Routes

Method	Endpoint	Auth?	Description
GET	/api/listings	No	Returns all available listings with seller info
POST	/api/listings	Yes	Creates a new listing. Body: title, description, price, category, condition, image_url, neighbourhood, delivery_option
GET	/api/listings/<id>	No	Returns a single listing by ID
DELETE	/api/listings/<id>	Yes	Soft-deletes listing (sets is_available = FALSE). Must be owner.
GET	/api/users/<id>/listings	No	Returns all active listings for a given user ID

5.3 AI Routes

Method	Endpoint	Auth?	Description
POST	/api/ai/scan-image	Yes	Sends image URL to Groq Vision. Returns { title, description, category, condition }
POST	/api/ai/suggest-price	Yes	Sends listing info to Groq text model. Returns { price_range: {min, max}, suggested_price, reasoning }

5.4 Messaging Routes

Method	Endpoint	Auth?	Description
GET	/api/conversations	Yes	Returns all conversations for the current user with last message and unread_count
POST	/api/conversations	Yes	Creates or retrieves a conversation with another user. Body: { other_user_id }
GET	/api/conversations/<id>/messages	Yes	Returns all messages in a conversation (chronological)
POST	/api/conversations/<id>/messages	Yes	Sends a message. Body: { text }
GET	/api/conversations/unread	Yes	Returns total unread message count across all conversations
POST	/api/conversations/<id>/read	Yes	Marks a conversation as read (upserts last_read_at timestamp)

6. Environment Variables & Configuration

6.1 Frontend (.env in project root)

VITE_ prefixed variables are injected by Vite at build time. They must be set as Docker build ARGs in Render.

Variable	Where to Find	Purpose
VITE_FIREBASE_API_KEY	Firebase Console → Project Settings → Your App → firebaseConfig.apiKey	Firebase client auth
VITE_FIREBASE_AUTH_DOMAIN	firebaseConfig.authDomain	Firebase auth domain
VITE_FIREBASE_PROJECT_ID	firebaseConfig.projectId	Firebase project ID
VITE_FIREBASE_STORAGE_BUCKET	firebaseConfig.storageBucket	Firebase storage bucket
VITE_FIREBASE_MESSAGING_SENDER_ID	firebaseConfig.messagingSenderId	Firebase messaging
VITE_FIREBASE_APP_ID	firebaseConfig.appId	Firebase app ID
VITE_CLOUDINARY_CLOUD_NAME	Cloudinary Dashboard (top of page)	Cloud name for uploads
VITE_CLOUDINARY_UPLOAD_PRESET	Cloudinary → Settings → Upload → Upload presets	Unsigned preset name
VITE_STORAGE_PROVIDER	Set to: cloudinary	Storage provider selector

6.2 Backend (backend/.env)

Variable	Where to Find	Purpose
DATABASE_URL	Supabase → Project Settings → Database → URI	PostgreSQL connection string
FLASK_SECRET_KEY	Make up any long random string	Flask session encryption key
GROQ_API_KEY	console.groq.com → API Keys	Groq AI API key
AI_PROVIDER	Set to: groq	AI provider selector (groq or aws_bedrock)

The backend also requires firebase-service-account.json in the backend/ folder. This file is downloaded from Firebase Console → Project Settings → Service Accounts → Generate new private key. It must never be committed to GitHub.

7. Services Used — Free Tier Limits & Constraints

7.1 Supabase (PostgreSQL Database)

Limit	Free Tier	Notes
Database Storage	500 MB	Enough for ~hundreds of thousands of listings and messages
Bandwidth	5 GB / month	Outgoing data transfer
Compute Hours	500 hours / month	Active query processing time
Inactivity Pause	Project pauses after 1 week of inactivity	Resume manually from Supabase dashboard — takes ~30 seconds
Connections	Up to 60 direct connections	Flask uses psycopg2 direct connections
Backups	Daily backups, 7-day retention	Download from Dashboard → Database → Backups
Projects	2 free projects	Current usage: 1

⚠️ Important: The free tier project pauses after 1 week of no activity. If users report the app being slow/down, go to Supabase dashboard and click Resume on the project.

7.2 Firebase Authentication

Limit	Free Tier (Spark Plan)	Notes
Monthly Active Users	50,000 MAU	More than sufficient for a campus marketplace
Email/Password Auth	Unlimited	No rate limits on sign-ups
ID Token Verification	Unlimited	No cost for server-side verification
Email Verification Emails	100/day via Firebase quota	Quota shared with password reset emails
Service Account Keys	Unlimited	The .json file used by Flask backend

7.3 Cloudinary (Image & Video Storage)

Limit	Free Tier	Notes
Storage	25 GB total	Images + video combined
Bandwidth	25 GB / month	CDN delivery to end users
Transformations	25 credits / month	1 credit = 1 image transformation (resize, crop)
File Size	10 MB per image, 100 MB per video	Enforced by Cloudinary

Limit	Free Tier	Notes
Upload Preset	Must be Unsigned for direct browser upload	Signed presets require backend involvement
Formats	All common image + video formats	JPEG, PNG, GIF, MP4, MOV, WebM, etc.

⚠️ Videos count significantly toward storage. If storage approaches 25 GB, consider adding image compression or limiting video uploads.

7.4 Groq (AI — Free Tier)

Limit	Free Tier	Notes
Requests per Day	14,400 requests/day	Shared across all Groq models
Requests per Minute	30 RPM (varies by model)	Llama 4 Scout: 30 RPM, Llama 3.1 8B: 30 RPM
Tokens per Minute	~6,000 TPM (varies)	Each AI scan/suggest uses ~300-500 tokens
Context Window	128K tokens (Scout), 128K (8B)	No concern for our use case
Cost	Free	No credit card required
Rate Limit on Error	Returns 429 status	ai_provider.py surfaces this as a 503 to the frontend

7.5 Render (Hosting)

Limit	Free Tier	Notes
Sleep on Inactivity	Service sleeps after 15 minutes of no traffic	First request after sleep takes 30-60 seconds (cold start)
Bandwidth	100 GB / month outbound	Should be sufficient for a campus app
Build Minutes	500 min / month	Each Docker build uses ~5-8 minutes
Custom Domains	1 free custom domain	Can add sellowl.com etc.
Environment Variables	Unlimited	Both runtime and build-time (ARG) vars supported
Docker Support	Yes (free)	Multi-stage builds supported

⚠️ The free tier sleep behavior means the first user to visit after 15 minutes of inactivity will experience a slow load. This is a known limitation of Render's free tier. Upgrading to the $7/month Starter plan disables sleep.

8. Local Development Setup

8.1 Prerequisites

- Node.js v20+ (download from nodejs.org — LTS version)
- Python 3.11+ (download from python.org — tick 'Add to PATH' on Windows)
- Git (download from git-scm.com)
- Homebrew (Mac only — install from brew.sh)

8.2 One-Time Setup

Step 1: Clone the repo

```
git clone https://github.com/kj0306/sellowl.git
cd sellowl
```

Step 2: Install frontend dependencies

```
npm install
```

Step 3: Set up Python virtual environment

```
cd backend
python3 -m venv venv
source venv/bin/activate          # Mac/Linux
# venv\Scripts\activate          # Windows
pip install -r requirements.txt
```

Step 4: Create environment files — see Section 6 for all variable values

```
# In sellowl/ root — create file .env with all VITE_ variables
# In sellowl/backend/ — create file .env with backend variables
```

Step 5: Place firebase-service-account.json in backend/

8.3 Daily Development Commands

Run both commands in separate terminal windows simultaneously:

Terminal 1 — Backend (Flask):

```
cd sellowl/backend
source venv/bin/activate
PORT=5001 python app.py
```

Terminal 2 — Frontend (Vite):

```
cd sellowl
npm run dev
```

Then open http://localhost:5173 in your browser.

Note: Port 5000 is blocked on macOS by AirPlay Receiver. Use PORT=5001 to avoid the conflict, or disable AirPlay Receiver in System Settings → General → AirDrop & Handoff.

8.4 Deploying to Production

```
git add -A
git commit -m "feat: your change description"
git push origin main
```

Render automatically detects the push and starts a new Docker build. Build takes approximately 4-6 minutes.

9. Future Improvements & Roadmap

9.1 Short-Term (Next Sprint)

- Orders & Offers system: Wire the existing Checkout.jsx and Offers.jsx components to the database. Create orders and offers tables. Allow sellers to accept/reject purchase requests.
- Real-time messaging with WebSockets: Replace 30-second polling with a WebSocket connection (Flask-SocketIO) for instant message delivery.
- Like/Save persistence: Persist liked listings to the database so they survive page refreshes and appear in a Saved Items tab.
- Push notifications: Use Firebase Cloud Messaging (FCM) to notify users of new messages and offer responses.

9.2 Medium-Term

- Image compression: Use Cloudinary's transformation pipeline to auto-resize and compress uploaded images before storing, reducing storage usage.
- Search improvements: Add full-text search using PostgreSQL's tsvector/tsquery for better search relevance.
- Listing categories expansion: Add Sublease/Housing as a separate zone with different fields (rent amount, lease dates, number of rooms).
- Seller ratings: After a completed transaction, allow buyers to rate sellers (1-5 stars with optional comment).
- Google Maps integration for neighbourhood: Replace free-text neighbourhood field with Google Maps Places Autocomplete for consistent location data.

9.3 Infrastructure Upgrades

- Migrate to Render Starter ($7/month): Eliminates the 15-minute sleep cold start. Essential before any public launch.
- AWS S3 for storage: When Cloudinary's 25 GB free tier approaches capacity, switch to AWS S3 by setting VITE_STORAGE_PROVIDER=s3 and implementing the presign route in the backend. The storage.js abstraction is already in place.
- AWS Bedrock for AI: To access Claude or Llama via AWS, set AI_PROVIDER=aws_bedrock, add boto3 to requirements.txt, and uncomment the AWSBedrockProvider class in ai_provider.py.
- Supabase Pro upgrade ($25/month): Removes the inactivity pause and increases storage to 8 GB. Enables point-in-time recovery backups.
- Rate limiting: Add Flask-Limiter to prevent API abuse, especially on AI routes which call paid external services.
- Monitoring with Sentry: Add Sentry SDK to both the Flask backend and React frontend for automatic error tracking and alerting.

9.4 Features Not Yet Started

Feature	Effort	Dependencies
Orders table + checkout flow	Medium	Requires orders DB table, backend routes
Offer accept/reject system	Medium	Requires offers DB table
Real-time chat (WebSocket)	Medium-High	Requires Flask-SocketIO, Render plan upgrade
Mobile app (React Native)	High	Reuse API, rebuild UI in React Native
University verification by domain	Low	Add allowed_domains config to firebase_auth.py
Bulk listing import (CSV)	Medium	New backend route + CSV parser
Listing expiry (auto-archive)	Low	Cron job or Supabase scheduled function
Multi-image listings	Medium	New listing_images table, update CreateListing UI
In-app payment (Stripe)	High	Stripe integration, escrow logic, compliance

10. Known Issues & Limitations

Issue	Impact	Fix / Workaround
Render free tier cold start (15 min sleep)	High — first load after inactivity is slow (~30-60s)	Upgrade to Render Starter ($7/month) or add UptimeRobot ping
Supabase free tier pauses after 1 week inactive	High — app breaks until manually resumed	Check Supabase dashboard, click Resume. Upgrade to Pro removes pause.
Python 3.8 on Mac (EOL)	Low — FutureWarning in local dev only	Run: brew install python@3.11, recreate venv
Port 5000 blocked by AirPlay on Mac	Low — local dev only	Use PORT=5001 python app.py
Messages not real-time (30s poll)	Medium — slight delay for recipient	Upgrade to WebSocket with Flask-SocketIO
Likes not persisted to DB	Low — lost on page refresh	Add a listing_likes table
No image compression on upload	Medium — large files slow the upload	Add Cloudinary upload transformation in storage.js
VITE_ vars require Docker rebuild to change	Low — Render redeploys on every push anyway	By design — Vite bakes vars at build time